Taller de experimentación con JMeter
“¿Qué pasaría si…?”

API Tiquetmania · carga, estrés y concurrencia

Pablo Guzmán
Automatización de Pruebas 2026-2
Noviembre de 2026

Repositorio: github.com/pabloandressoftware/tiquetmania

Cómo se hicieron las pruebas

Las 10 preguntas se probaron con un solo archivo, pruebas/experimentacion/Experimentacion.jmx. No se modifica el plan del despliegue (pruebas/jmeter/Ticketmania.jmx). El plan se configura con propiedades de JMeter (-Jusuarios, -Jrampup, -Jduracion, -Jflujo, -Jtimer_const, -Jsync...), así cada escenario es una ejecución distinta del mismo archivo. Cada ejecución se hizo en modo no-GUI, con la API recién iniciada, y generó su reporte HTML.

Qué contiene el plan

- Configuración del CSV Data Set con datos.csv: 32 compras (tarifa, cantidad, total_esperado). El total esperado es precio × cantidad.
- Valores por defecto de la petición HTTP (127.0.0.1:8080) y Gestor de Cabecera con Accept y Content-Type JSON.
- Flujo: Tarifas (GET) → SeleccionTarifaYCantidad (POST, JSON Extractor de num_reserva) → Pago (POST). Un controlador If permite probar solo Tarifas, solo Selección o el flujo completo.
- Aserciones: código 201/200, estado APROBADO y total = precio × cantidad (JSON Assertion contra la columna total_esperado).
- Temporizador Aleatorio Uniforme (500 ms + hasta 1000 ms por defecto) y Temporizador sincronizado opcional (pregunta 8).
- Cuatro Grupos de Hilos: uno principal y tres de escalones que se suman con un retraso (pregunta 4). Un grupo con 0 hilos no corre.

Equipo y limitaciones generales

Limitación: Todo corrió en mi computador: Ubuntu Linux, 16 núcleos y 31 GB de RAM, Java 21, JMeter 5.6.3 y la API de la profesora (Python, un solo proceso). JMeter y la API comparten el equipo, así que el resultado mide a los dos juntos. Esa es la principal limitación y aplica a todas las preguntas.

- Para CPU y memoria no usé PerfMon con ServerAgent. Escribí un muestreador propio (scripts/correr_experimentos.py, librería psutil) que anota CPU y memoria cada segundo del equipo completo, de JMeter y de la API. Mide lo mismo que PerfMon, pero no es el plugin que pide la guía.
- Para la pregunta 4 usé cuatro Grupos de Hilos estándar con retraso de inicio en lugar del Ultimate Thread Group, que es un plugin. El resultado es el mismo escalonado: 50, 100, 250 y 500 usuarios.
- Los valores son de una sola ejecución por escenario. Son una referencia, no un promedio de varias repeticiones.
- Las capturas de los reportes son del reporte HTML de JMeter. La captura del escenario es una tabla con los parámetros exactos de cada ejecución.

Resumen de los 18 escenarios

Escenario	Muestras	Media (ms)	p95 (ms)	p99 (ms)	Máx (ms)	% Error	Rendimiento (peticiones/s)
P1 · 20 usuarios	1 096	2.2	3	4	26	0.00 %	18.6
P1 · 500 usuarios	27 012	1.5	2	3	1 067	0.00 %	454.3
P2 · subida 1 s	17 775	1.5	2	3	1 042	0.00 %	198.8
P2 · subida 60 s	11 913	1.5	2	3	35	0.00 %	134.4
P3 · con temporizador	5 417	1.7	2	3	23	0.00 %	91.5
P3 · sin temporizador	160 596	35.7	3	1 052	108 793	0.00 %	1 295.8
P4 · escalones	172 435	48.2	3	1 255	35 839	0.00 %	1 271.2
P5 · 1 minuto	5 458	1.7	2	3	32	0.00 %	92.3

P5 · 10 minutos	59 469	1.4	2	2	29	0.00 %	99.3
P6 · /api/tarifas	54 464	1.3	2	3	1 055	0.00 %	910.2
P6 · /api/seleccion	54 373	1.6	2	3	1 232	0.00 %	909.0
P7 · 500 reservas a la vez	5 000	1 489.3	4 119	50 426	105 619	0.66 %	43.9
P8 · sincronizado	500	28 442.3	126 253	126 266	126 273	16.60 %	4.0
P8 · distribuido	500	25.8	4	1 099	1 302	0.00 %	51.5
P9 · totales bajo carga	54 728	1.4	2	3	1 059	0.00 %	914.6
P9 · control negativo	120	3.7	7	40	40	33.33 %	151.7
P10 · 500 usuarios	81 291	1.2	2	3	1 063	0.00 %	453.3
P10 · 500 con pausas cortas	407 796	103.7	1 022	2 074	135 385	0.00 %	2 005.8

El rendimiento de esta tabla es peticiones ÷ tiempo desde el primer envío hasta la última respuesta. En los escenarios con respuestas muy lentas al final (P3 sin temporizador, P4, P7, P8) esa cuenta se baja porque la prueba se alarga; en cada pregunta uso el rendimiento de los primeros 60 s cuando hace falta.

Pregunta 1 · Carga

¿Qué pasaría si… *subo de 20 a 500 usuarios? ¿Cambia el throughput, el tiempo de respuesta o ambos?*

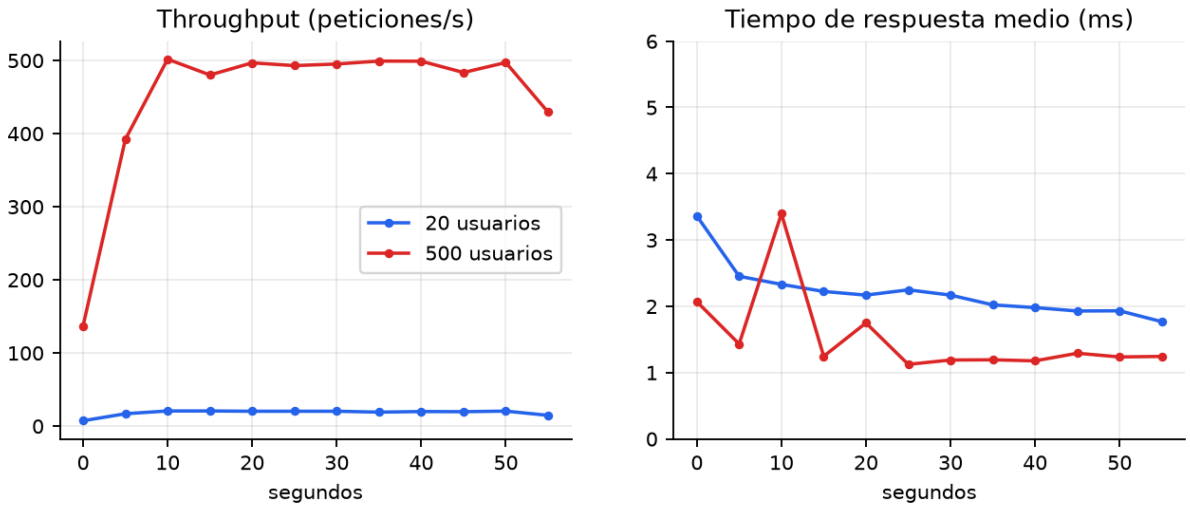
Escenario

Parámetro	Valor
Hilos (usuarios)	20 y 500 (dos ejecuciones)
Periodo de subida	10 s
Duración	60 s (bucle infinito, scheduler 60 s)
Temporizador	Aleatorio Uniforme: 500 ms + hasta 1000 ms
Flujo	Tarifas → Selección → Pago
Propiedades	-Jusuarios=20|500 -Jrampup=10 -Jduracion=60 -Jiteraciones=-1

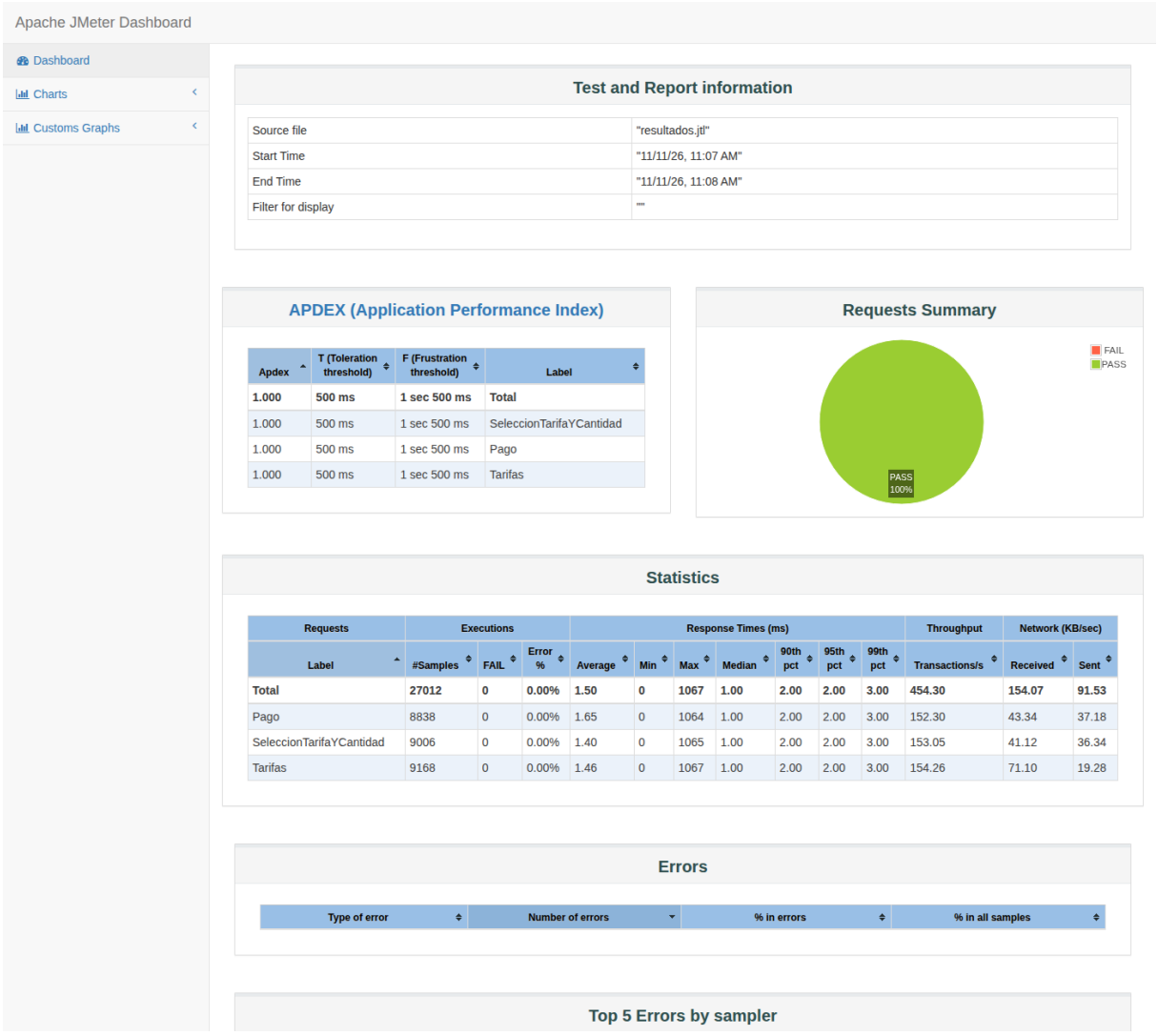
Configuración del escenario de la pregunta 1.

Resultados observados

Escenario	Muestras	Media (ms)	p95 (ms)	p99 (ms)	Máx (ms)	% Error	Rendimiento (peticiones/s)
20 usuarios	1 096	2.2	3	4	26	0.00 %	18.6
500 usuarios	27 012	1.5	2	3	1 067	0.00 %	454.3



Throughput y tiempo de respuesta cada 5 s. El throughput se multiplica por 25; el tiempo de respuesta no cambia.



Reporte HTML de JMeter, 500 usuarios.

Mi respuesta

Cambia el throughput y el tiempo de respuesta se mantiene. Con 20 usuarios la API completó unas 20 peticiones por segundo y con 500 usuarios unas 495. Son 25 veces más usuarios y casi 25 veces más peticiones. El tiempo medio de respuesta quedó en 2.2 ms con 20 usuarios y 1.5 ms con 500, y el p95 en 3 y 2 ms.

Esto pasa porque 500 usuarios con pausas de 1 s casi no presionan a la API: ella puede con muchas más peticiones por segundo (la pregunta 4 muestra que aguanta unas 2 300). El único cambio visible está en el máximo: con 500 usuarios hay unas pocas peticiones de cerca de 1 segundo (1067 ms) mientras suben los usuarios. Esas pocas peticiones no mueven el promedio. Mi hipótesis es que son conexiones nuevas que el servidor no alcanzó a aceptar y se reintentaron al segundo, pero no lo comprobé.

Qué no pude medir o qué cambió el resultado

- Los dos escenarios usan pausas, así que la carga está controlada por los temporizadores. Sin pausas el resultado sería otro (pregunta 3).
- API y JMeter comparten el equipo: con 500 usuarios JMeter usó hasta 3,9 núcleos y la API apenas 0,5.

Pregunta 2 · Carga

¿Qué pasaría si… *el ramp-up pasa de 1 segundo a 60 segundos? ¿Cambia el pico inicial del reporte?*

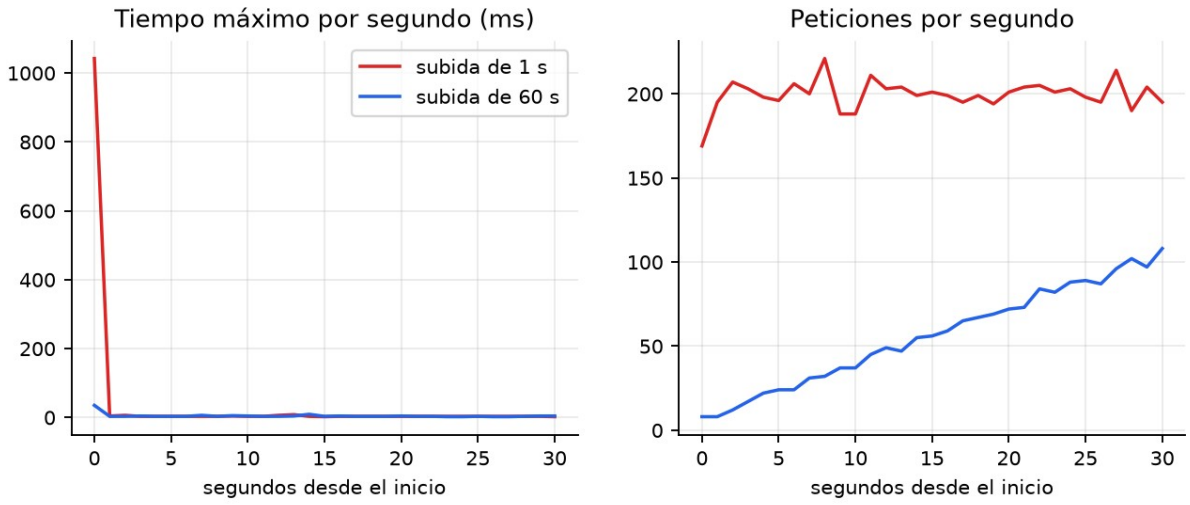
Escenario

Parámetro	Valor
Hilos (usuarios)	200
Periodo de subida	1 s y 60 s (dos ejecuciones)
Duración	90 s (bucle infinito)
Temporizador	Aleatorio Uniforme: 500 ms + hasta 1000 ms
Flujo	Tarifas → Selección → Pago
Propiedades	-Jusuarios=200 -Jrampup=1|60 -Jduracion=90 -Jiteraciones=-1

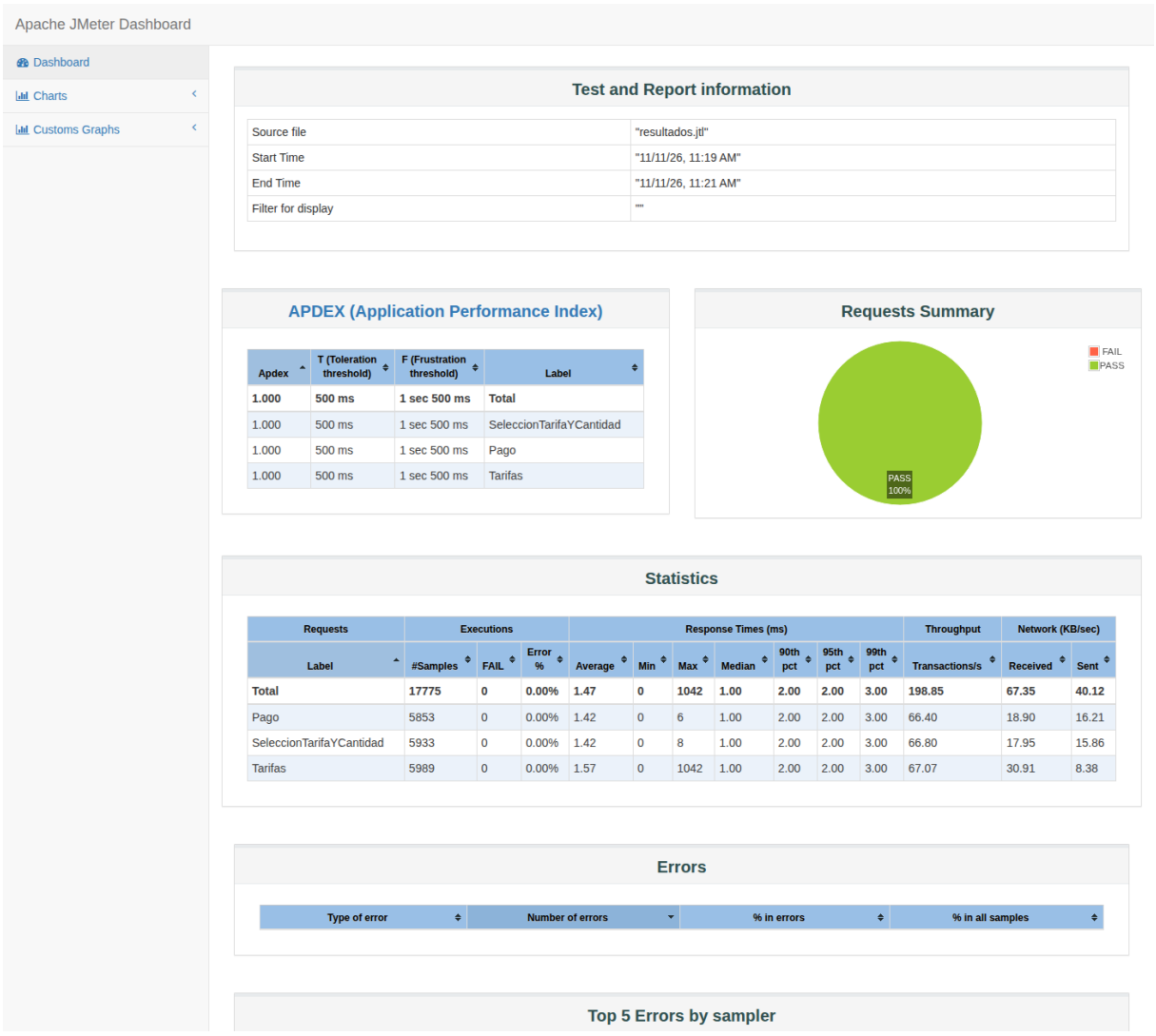
Configuración del escenario de la pregunta 2.

Resultados observados

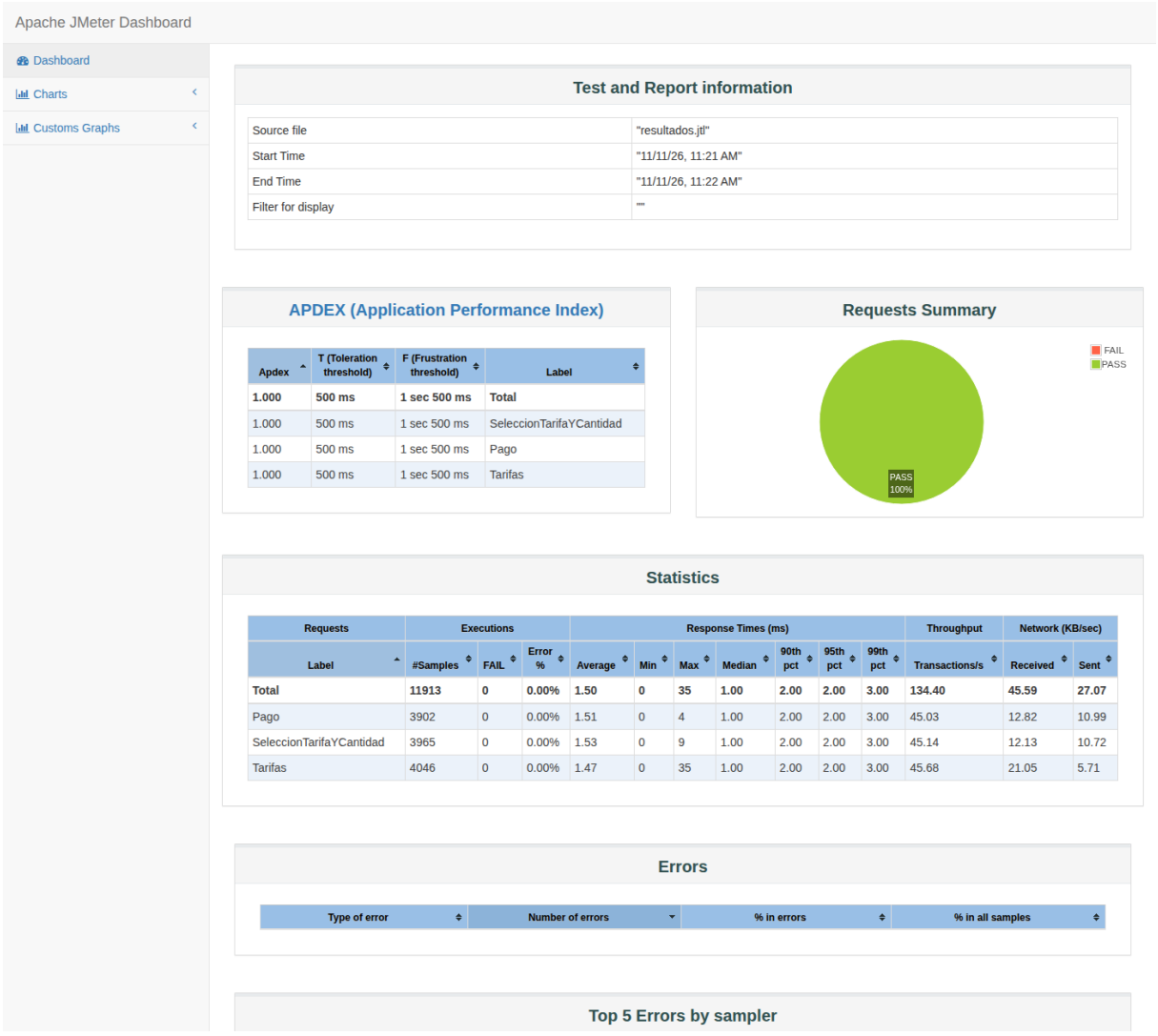
Escenario	Máx. en los primeros 5 s (ms)	Peticiones > 500 ms en los primeros 5 s	Media (ms)	p95 (ms)	Máx. total (ms)	% Error
Subida de 1 s	1042	1	1.5	2	1042	0.00 %
Subida de 60 s	35	0	1.5	2	35	0.00 %



Primeros 30 s. Con subida de 1 s el máximo salta a más de 1 s al inicio; con 60 s la carga sube de forma gradual.



Reporte de JMeter, subida de 1 s.



Reporte de JMeter, subida de 60 s.

Mi respuesta

Sí, cambia el pico inicial, pero es un pico de pocas peticiones. Con subida de 1 s los 200 usuarios llegan casi juntos: la carga sube de golpe a 198 peticiones por segundo y en los primeros 5 segundos aparece un pico: 1 petición de más de medio segundo (la más lenta, 1042 ms). Con subida de 60 s la carga sube en rampa, de unas 21 a unas 190 peticiones por segundo, y el máximo de los primeros 5 s es de 35 ms.

Después de ese arranque los dos escenarios se comportan igual: media de 1,5 ms y p95 de 2 ms. La subida corta no hace que la API falle, solo hace visible el momento de arranque. Con subida larga ese pico desaparece, pero también tarda cerca de un minuto en llegar a la carga completa.

Qué no pude medir o qué cambió el resultado

- En el reporte el pico se ve en la gráfica de tiempos de respuesta en el tiempo, no en el promedio general.
- No sé exactamente qué causa el pico de 1 segundo (puede ser el reintento de conexiones nuevas). No lo medí con herramientas de red.

Pregunta 3 · Carga

¿Qué pasaría si… *quito los temporizadores? ¿Cuánto sube el throughput y qué significa eso para la prueba?*

Escenario

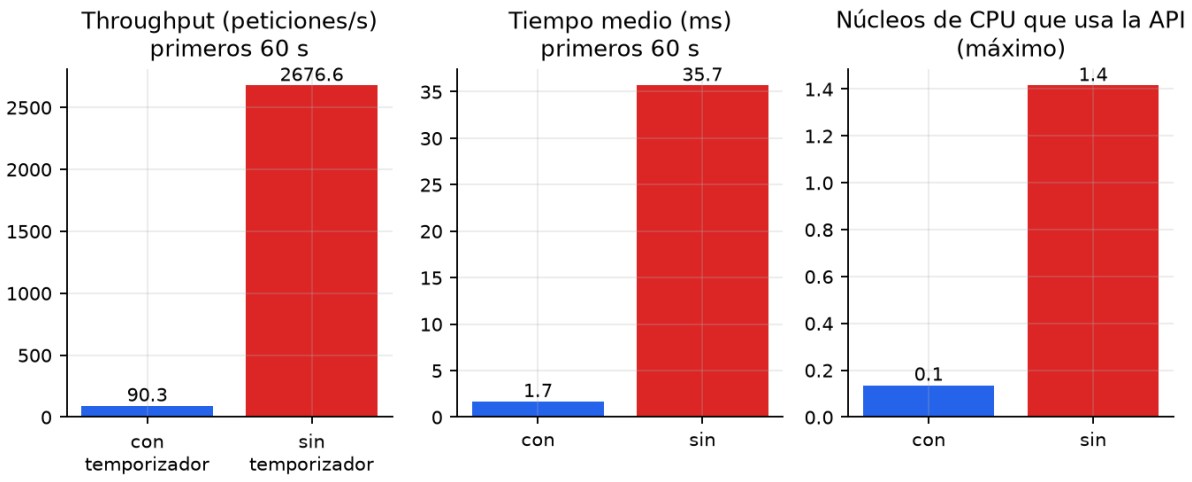
Parámetro	Valor
Hilos (usuarios)	100
Periodo de subida	10 s
Duración	60 s (bucle infinito)
Temporizador	con: 500 ms + hasta 1000 ms / sin: 0 ms + 0 ms
Flujo	Tarifas → Selección → Pago
Propiedades	-Jusuarios=100 -Jrampup=10 -Jduracion=60 -Jtimer_const=0 -Jtimer_rand=0

Configuración del escenario de la pregunta 3.

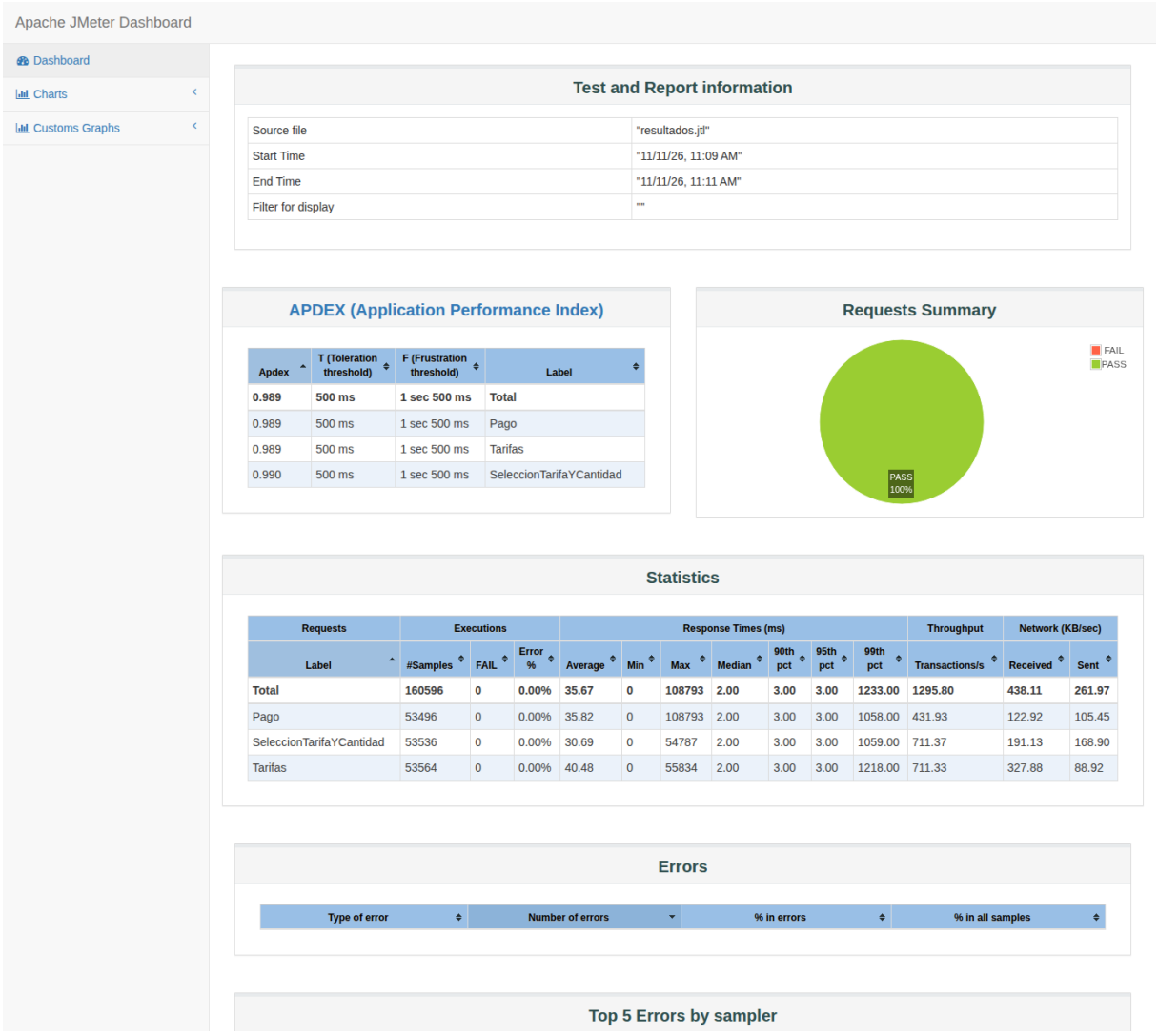
Resultados observados

Escenario	Muestras	Media (ms)	p95 (ms)	p99 (ms)	Máx (ms)	% Error	Rendimiento (peticiones/s)
Con temporizador	5 417	1.7	2	3	23	0.00 %	91.5
Sin temporizador	160 596	35.7	3	1 052	108 793	0.00 %	2 676.6

Escenario (primeros 60 s)	Rendimiento (peticiones/s)	Media (ms)	CPU máx. de la API (núcleos)
Con temporizador	90	1.7	0.14
Sin temporizador	2677	35.7	1.42



Sin temporizadores el throughput se multiplica por unas 30 y la API llega a usar más de un núcleo completo.



Reporte de JMeter sin temporizadores: los máximos llegan a más de 100 s.

Mi respuesta

Sin temporizadores el throughput sube de unas 90 a unas 2677 peticiones por segundo: unas 30 veces más con los mismos 100 usuarios. El tiempo medio también sube (de 1.7 ms a 36 ms), el p99 pasa a 1052 ms y el máximo llega a 109 segundos.

Lo que significa para la prueba: los temporizadores representan el tiempo que una persona real tarda en pensar entre un clic y el siguiente. Sin ellos, cada usuario virtual dispara una petición apenas termina la anterior, y 100 usuarios generan la presión de unas 3000 personas reales. Ya no es una prueba de carga normal, es una prueba de estrés: mide hasta dónde llega la API y no cómo la usarían los clientes.

También se nota que la API empieza a saturarse: usa más de un núcleo (141 %), el tiempo de algunas peticiones se dispara y la prueba de 60 s terminó a los 124 s porque había respuestas pendientes.

Qué no pude medir o qué cambió el resultado

- Con temporizadores o sin ellos, ninguna petición falló: el límite aparece como demora, no como error.

- El rendimiento general del reporte (1 296 por segundo) está por debajo del real porque la prueba se alargó por las respuestas lentas del final. Por eso en la tabla uso los primeros 60 s.
- JMeter y la API compiten por el mismo equipo, así que parte de la demora puede venir de JMeter y no solo de la API.

Pregunta 4 · Estrés

¿Qué pasaría si… *escalono de 50 a 100, 250 y 500 usuarios? ¿En qué punto deja de crecer el throughput?*

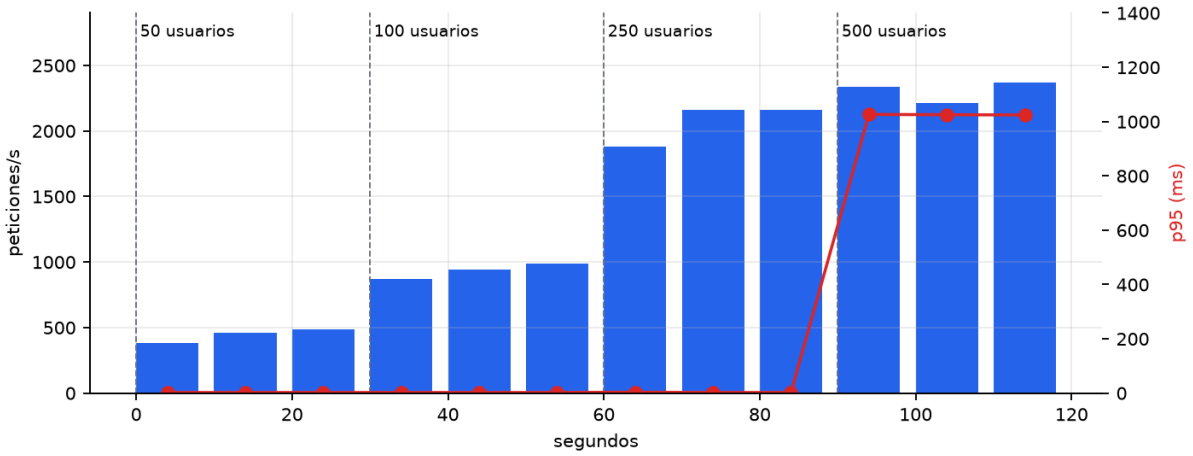
Escenario

Parámetro	Valor
Escalón 1 (desde 0 s)	50 usuarios (subida 5 s)
Escalón 2 (desde 30 s)	+50 → 100 usuarios
Escalón 3 (desde 60 s)	+150 → 250 usuarios
Escalón 4 (desde 90 s)	+250 → 500 usuarios
Duración total	120 s (cada escalón ~30 s)
Temporizador	50 ms + hasta 100 ms (corto, para empujar la carga)
Flujo	Tarifas → Selección → Pago
Propiedades	-Jusuarios=50 -Jesc2=50 -Jesc2_retraso=30 -Jesc3=150 -Jesc3_retraso=60 -Jesc4=250 -Jesc4_retraso=90

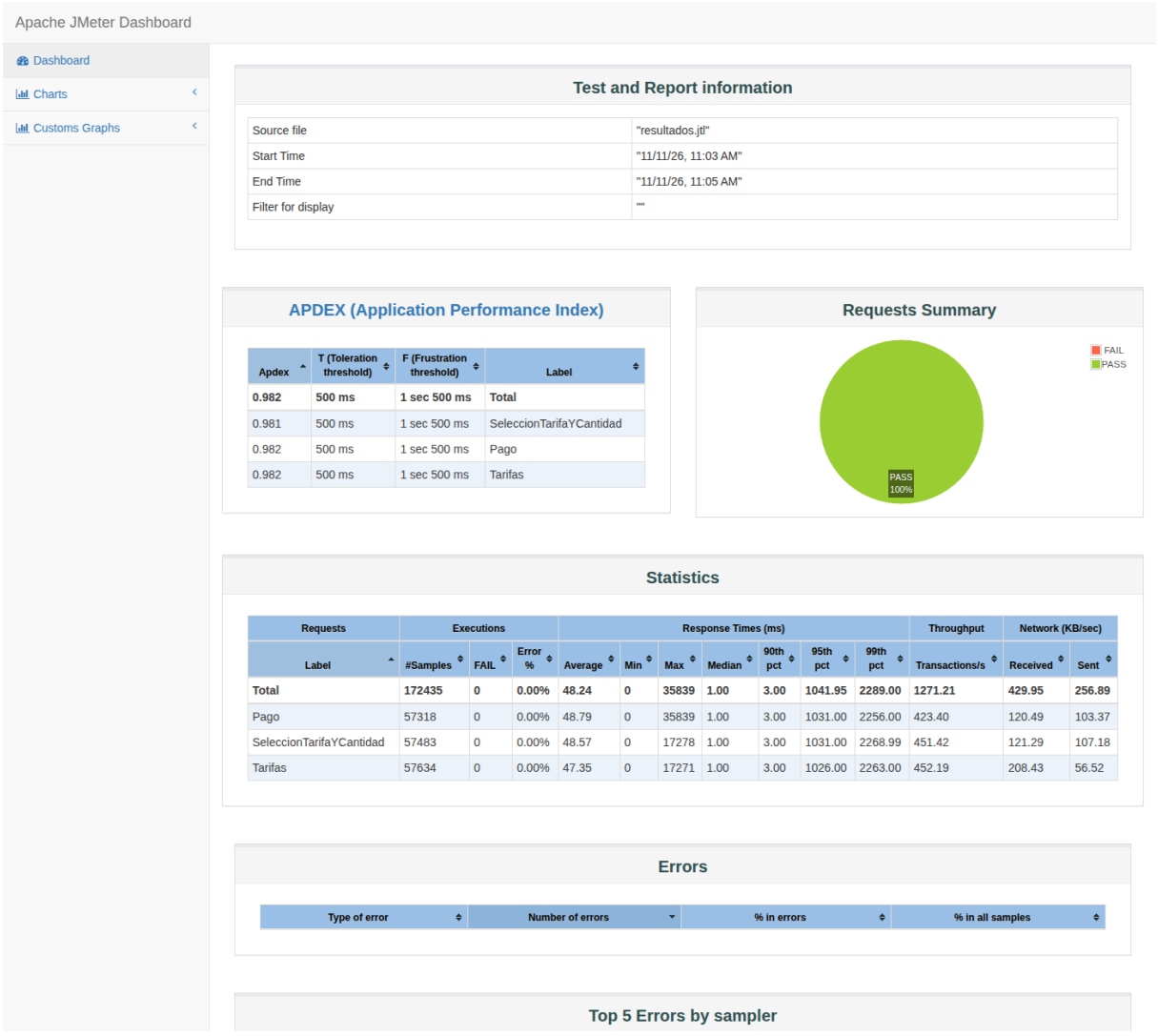
Configuración del escenario de la pregunta 4: cuatro Grupos de Hilos con retraso de inicio.

Resultados observados

Escalón (usuarios)	Ventana	Rendimiento (peticiones/s)	Media (ms)	p95 (ms)	Máx. (ms)
50	10–20 s	458.2	1.3	2	5
100	40–50 s	943.5	1.0	2	1039
250	70–80 s	2157.8	16.6	2	2080
500	100–110 s	2215.2	111.0	1025	17402



Throughput por ventana de 10 s (barras) y p95 (línea roja). El throughput se aplana entre 250 y 500 usuarios.



Reporte de JMeter del escenario escalonado.

Mi respuesta

Deja de crecer entre 250 y 500 usuarios, cerca de las 2 300 peticiones por segundo. Con 50 usuarios el throughput fue de unas 458 por segundo, con 100 de unas 944, con 250 de unas 2158 y con 500 de unas 2306. Pasar de 50 a 100 y de 100 a 250 usuarios casi duplicó el throughput cada vez. Pasar de 250 a 500 lo subió solo un 7 %, aunque se duplicaron los usuarios.

Cuando el throughput se estanca, el tiempo de respuesta se rompe: el p95 pasa de 2 ms a más de 1 000 ms con 500 usuarios y el máximo llega a 36 segundos. Es la señal clásica de saturación: llegan más peticiones de las que la API puede atender, entonces se acumulan en fila. Con 250 usuarios ya se ve el comienzo (el promedio sube de 1 a 16 ms).

Qué no pude medir o qué cambió el resultado

- Tampoco hubo errores: la saturación se ve como demora, no como fallas.
- El límite de unas 2 300 peticiones por segundo es del conjunto API y JMeter en mi equipo; la API es un solo proceso de Python, lo que probablemente limita este número. No probé otra máquina.

- Usé temporizadores cortos (50 a 150 ms). Con las pausas por defecto 500 usuarios solo generan unas 500 peticiones por segundo y la API no llegaría a saturarse.

Pregunta 5 · Estrés

¿Qué pasaría si… *la prueba dura 10 minutos en lugar de 1? ¿El tiempo de respuesta se mantiene o sube con el tiempo?*

Escenario

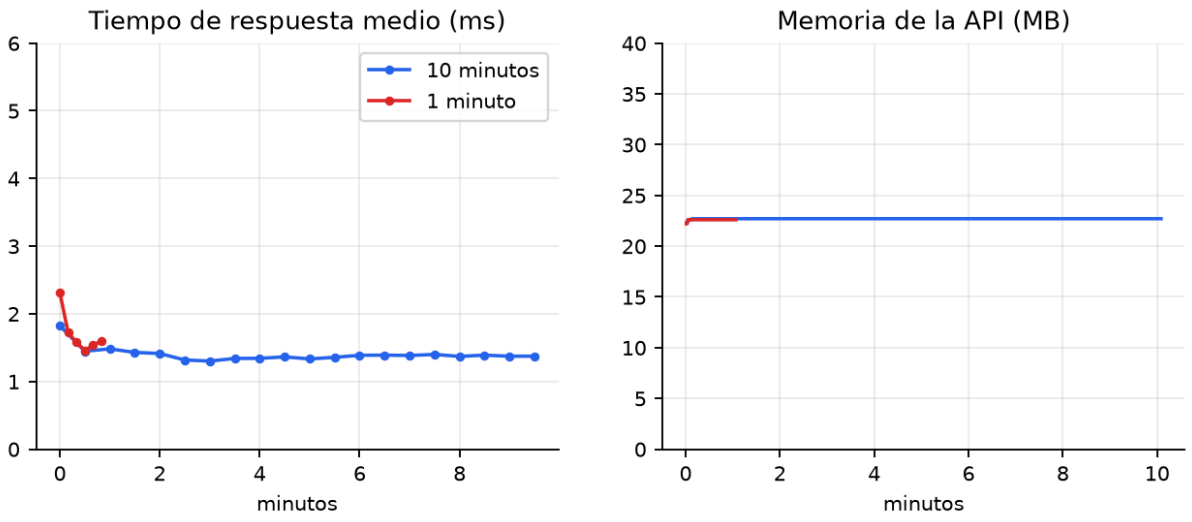
Parámetro	Valor
Hilos (usuarios)	100
Periodo de subida	10 s
Duración	60 s y 600 s (dos ejecuciones)
Temporizador	Aleatorio Uniforme: 500 ms + hasta 1000 ms
Flujo	Tarifas → Selección → Pago
Propiedades	-Jusuarios=100 -Jrampup=10 -Jduracion=60|600 -Jiteraciones=-1

Configuración del escenario de la pregunta 5.

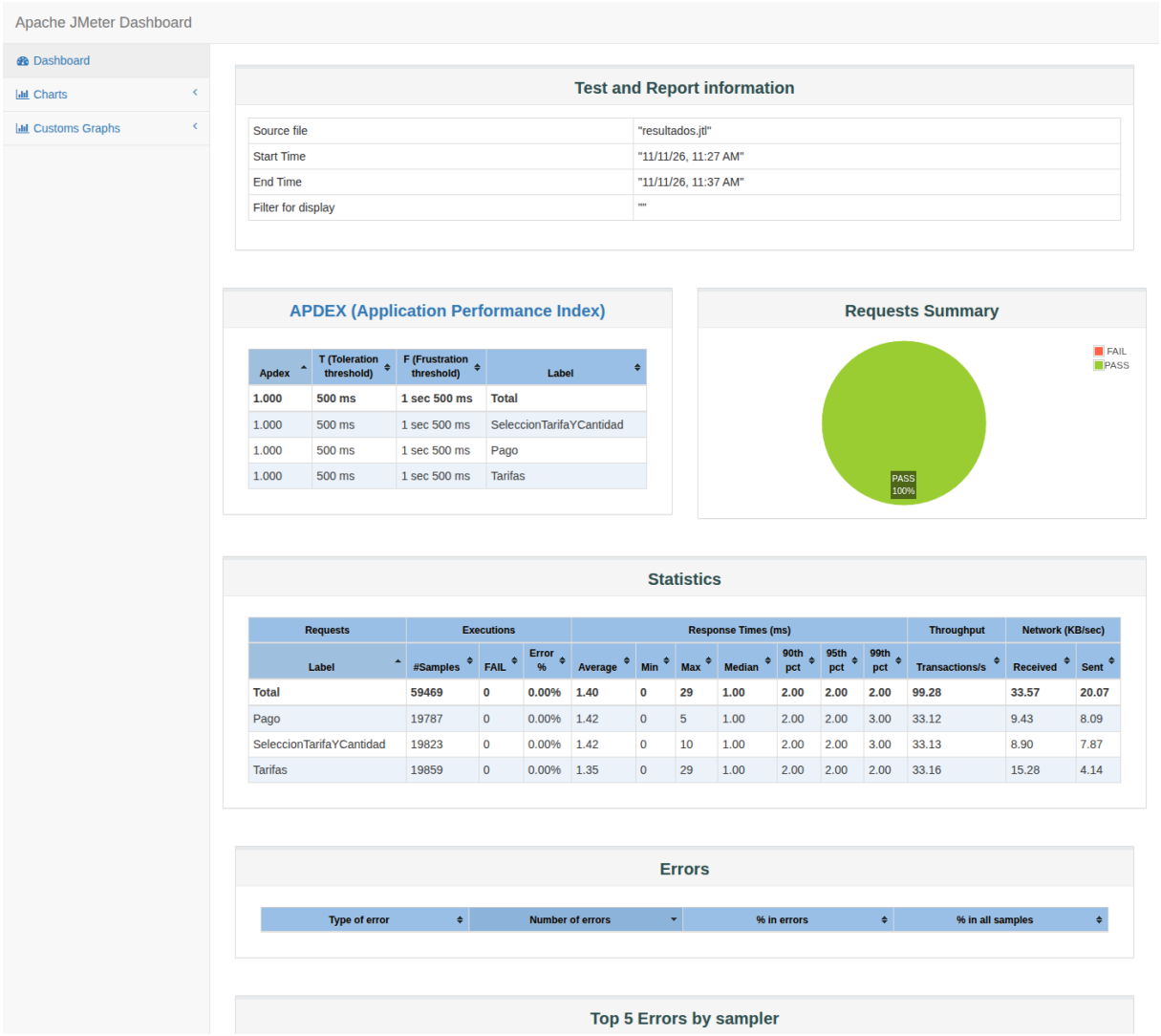
Resultados observados

Escenario	Muestras	Media (ms)	p95 (ms)	p99 (ms)	Máx (ms)	% Error	Rendimiento (peticiones/s)
1 minuto	5 458	1.7	2	3	32	0.00 %	92.3
10 minutos	59 469	1.4	2	2	29	0.00 %	99.3

Minuto	1	2	3	5	7	10
Media (ms)	1.63	1.47	1.35	1.35	1.4	1.4
p95 (ms)	3	2	2	2	2	2



Tiempo medio a lo largo de la prueba (izquierda) y memoria de la API (derecha).



Reporte de JMeter de la prueba de 10 minutos.

Mi respuesta

Se mantiene. La media fue de 1.7 ms en la prueba de 1 minuto y de 1.4 ms en la de 10 minutos; el p95 fue de 2 ms en las dos. Minuto a minuto el tiempo se queda plano, sin tendencia a subir. Tampoco crece la memoria de la API: pasó de 22,3 a 22,7 MB en 10 minutos.

Tiene sentido porque la carga es moderada (unas 100 peticiones por segundo) y cada reserva se borra al pagarse. No hay nada que se acumule. El único valor más alto está en los primeros segundos (máximo de 29 ms), cuando están arrancando los hilos.

Qué no pude medir o qué cambió el resultado

- Con esta carga 10 minutos no alcanzan para ver un problema. Para buscar fugas de memoria o degradación lenta haría falta una prueba más larga o con más carga.
- El caso contrario sí aparece en la pregunta 6: las reservas que no se pagan se acumulan en memoria.

Pregunta 6 · Estrés

¿Qué pasaría si… *mido /api/tarifas (solo lectura) y /api/seleccion (crea reservas) con la misma carga? ¿Cuál cuesta más?*

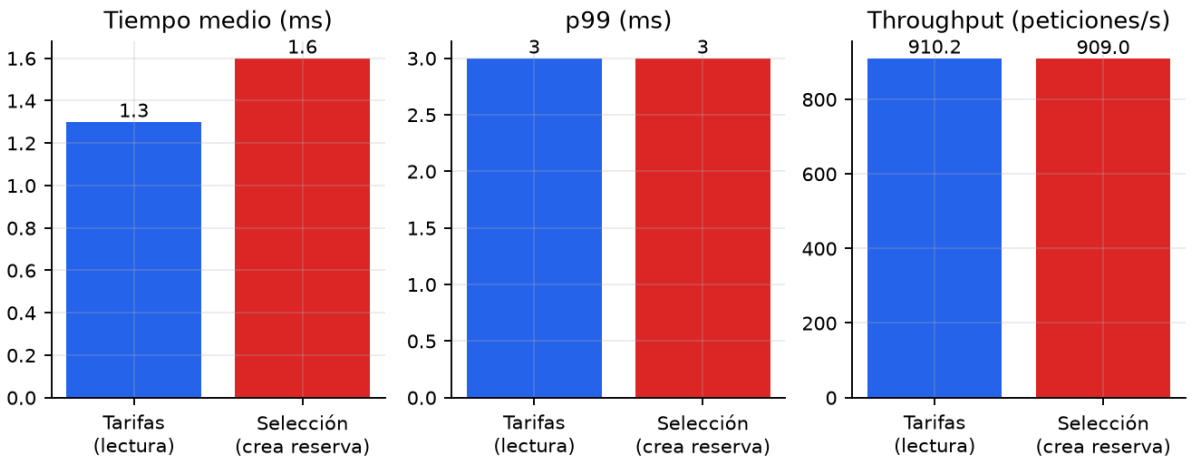
Escenario

Parámetro	Valor
Hilos (usuarios)	200
Periodo de subida	10 s
Duración	60 s (bucle infinito)
Temporizador	100 ms + hasta 200 ms
Petición medida	solo GET /api/tarifas, o solo POST /api/seleccion
Propiedades	-Jflujo=tarifas|seleccion -Jusuarios=200 -Jtimer_const=100 -Jtimer_rand=200

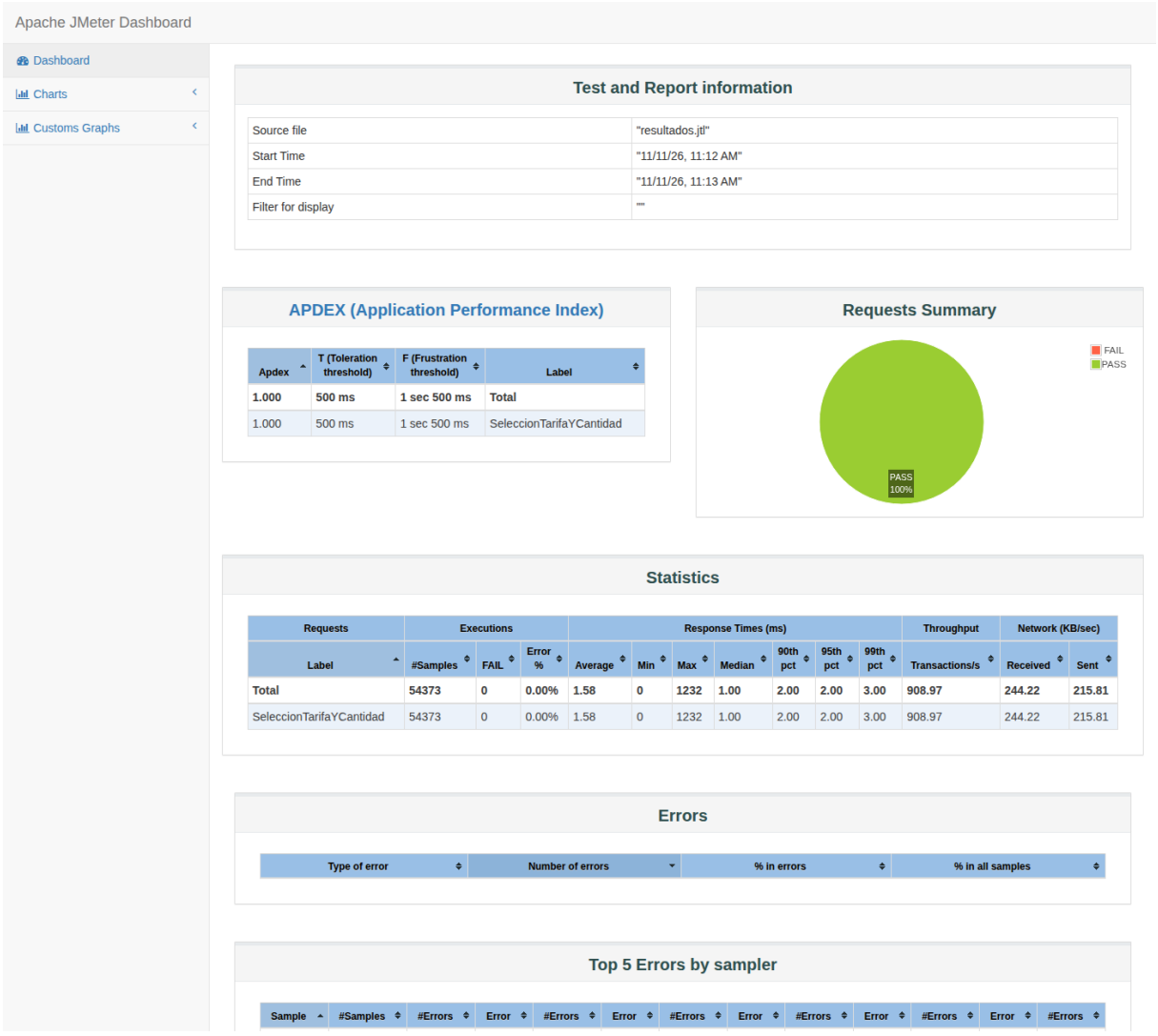
Configuración del escenario de la pregunta 6.

Resultados observados

Petición	Muestras	Media (ms)	p95 (ms)	p99 (ms)	Máx (ms)	Rendimiento (peticiones/s)	Memoria de la API al final (MB)
GET /api/tarifas	54464	1.3	2	3	1055	910.2	22.7
POST /api/seleccion	54373	1.6	2	3	1232	909.0	43.6



Costo (tiempo de respuesta) y rendimiento de las dos peticiones.



Reporte de JMeter de /api/seleccion.

Mi respuesta

Cuesta más /api/seleccion, pero por muy poco. Su tiempo medio fue de 1.6 ms contra 1.3 ms de /api/tarifas, o sea unos 0.3 ms más (cerca de un 23 % más). El p95 es igual (2 ms en las dos) y el rendimiento también (unas 910 peticiones por segundo), porque lo fijan los temporizadores y no la API.

Es lo esperado: /api/tarifas solo devuelve una lista fija y /api/seleccion además valida los datos genera un identificador y guarda la reserva en memoria. Ese guardado tiene otro costo que no se ve en el tiempo: como en este escenario nadie paga la memoria de la API pasó de 22.2 a 43.6 MB con 54 373 reservas y con /api/tarifas se quedó en 22.7 MB.

Qué no pude medir o qué cambió el resultado

- Con el costo medido como tiempo de respuesta, la diferencia de 0,3 ms es pequeña y está cerca de lo que mi equipo puede distinguir. Con una carga más alta se vería mejor.
- El costo en memoria no es el que pide la guía (tiempo de respuesta), pero lo anoto porque es una diferencia real entre las dos peticiones.

Pregunta 7 · Concurrencia

¿Qué pasaría si… *500 usuarios crean reservas al mismo tiempo? ¿Todas las respuestas son exitosas?*

Escenario

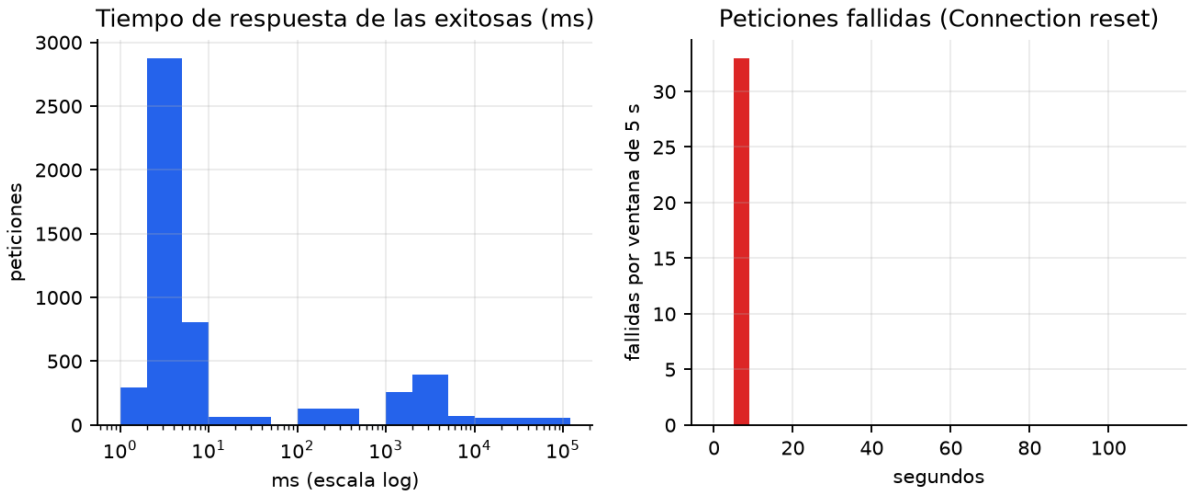
Parámetro	Valor
Hilos (usuarios)	500
Periodo de subida	1 s
Bucle	10 repeticiones por usuario (5 000 reservas)
Temporizador	ninguno (0 ms)
Petición	solo POST /api/seleccion
Aserciones	código 201 y total = precio × cantidad
Propiedades	-Jflujo=seleccion -Jusuarios=500 -Jrampup=1 -Jiteraciones=10 -Jtimer_const=0 -Jtimer_rand=0

Configuración del escenario de la pregunta 7.

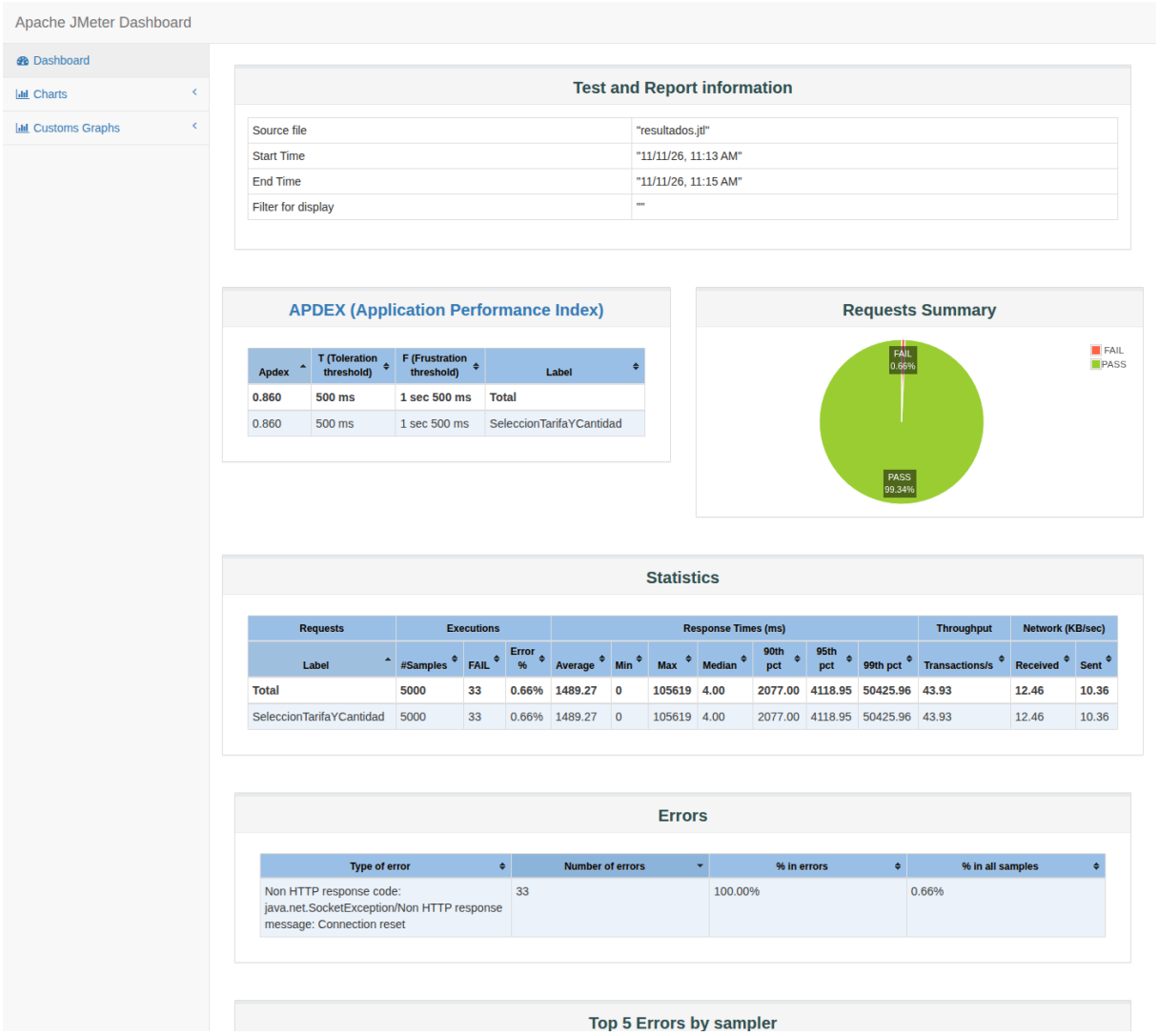
Resultados observados

Escenario	Muestras	Media (ms)	p95 (ms)	p99 (ms)	Máx (ms)	% Error	Rendimiento (peticiones/s)
500 reservas a la vez	5 000	1 489.3	4 119	50 426	105 619	0.66 %	43.9

Dato	Valor
Respuestas exitosas	4967 de 5 000
Respuestas fallidas	33 (todas: Non HTTP response code: java.net.SocketException / Connection reset)
Mediana de las exitosas	4 ms
p95 de las exitosas	3733 ms
Máximo	105619 ms



Distribución del tiempo de respuesta (izquierda) y errores en el tiempo (derecha).



Reporte de JMeter: aparecen errores y tiempos muy altos.

Mi respuesta

No, no todas fueron exitosas. De 5 000 peticiones, 33 (0.66 %) fallaron con “Connection reset”: el servidor cortó la conexión. Entre las que sí respondieron, la mayoría fue rápida (mediana de 4 ms), pero el p95 fue de 3.7 segundos y hubo respuestas de más de 100 segundos.

Lo importante es qué falló y qué no. Las reservas que se crearon fueron correctas: todas pasaron la validación de total (precio × cantidad) y devolvieron 201. Lo que no aguantó fue la llegada simultánea de conexiones: la API no alcanza a aceptarlas todas a la vez, unas esperan varios segundos y otras se rechazan. Mi hipótesis es que se llena la fila de conexiones pendientes del servidor, pero no lo comprobé con herramientas de red.

Qué no pude medir o qué cambió el resultado

- Que el tiempo total sea de casi 2 minutos con 5 000 peticiones sin pausas confirma que el problema es la llegada en bloque, no la cantidad.
- Un servidor real tendría una fila de conexiones más grande y quizá un balanceador. Esta API es un solo proceso de Python con la configuración básica.

Pregunta 8 · Concurrencia

¿Qué pasaría si… *los usuarios disparan a la vez con el Temporizador sincronizado, frente a cuando llegan distribuidos?*

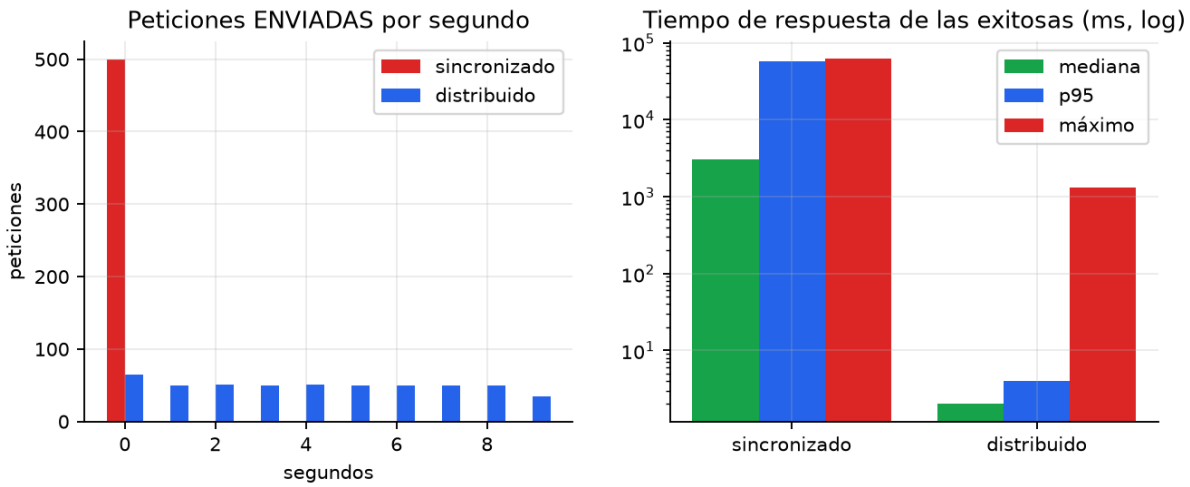
Escenario

Parámetro	Valor
Hilos (usuarios)	500, una sola petición cada uno (POST /api/seleccion)
Sincronizado	Temporizador sincronizado, grupo de 500; subida 5 s
Distribuido	sin temporizador sincronizado; subida 10 s
Temporizador aleatorio	ninguno (0 ms)
Propiedades	-Jsync=true -Jsync_grupo=500 -Jrampup=5 / -Jrampup=10

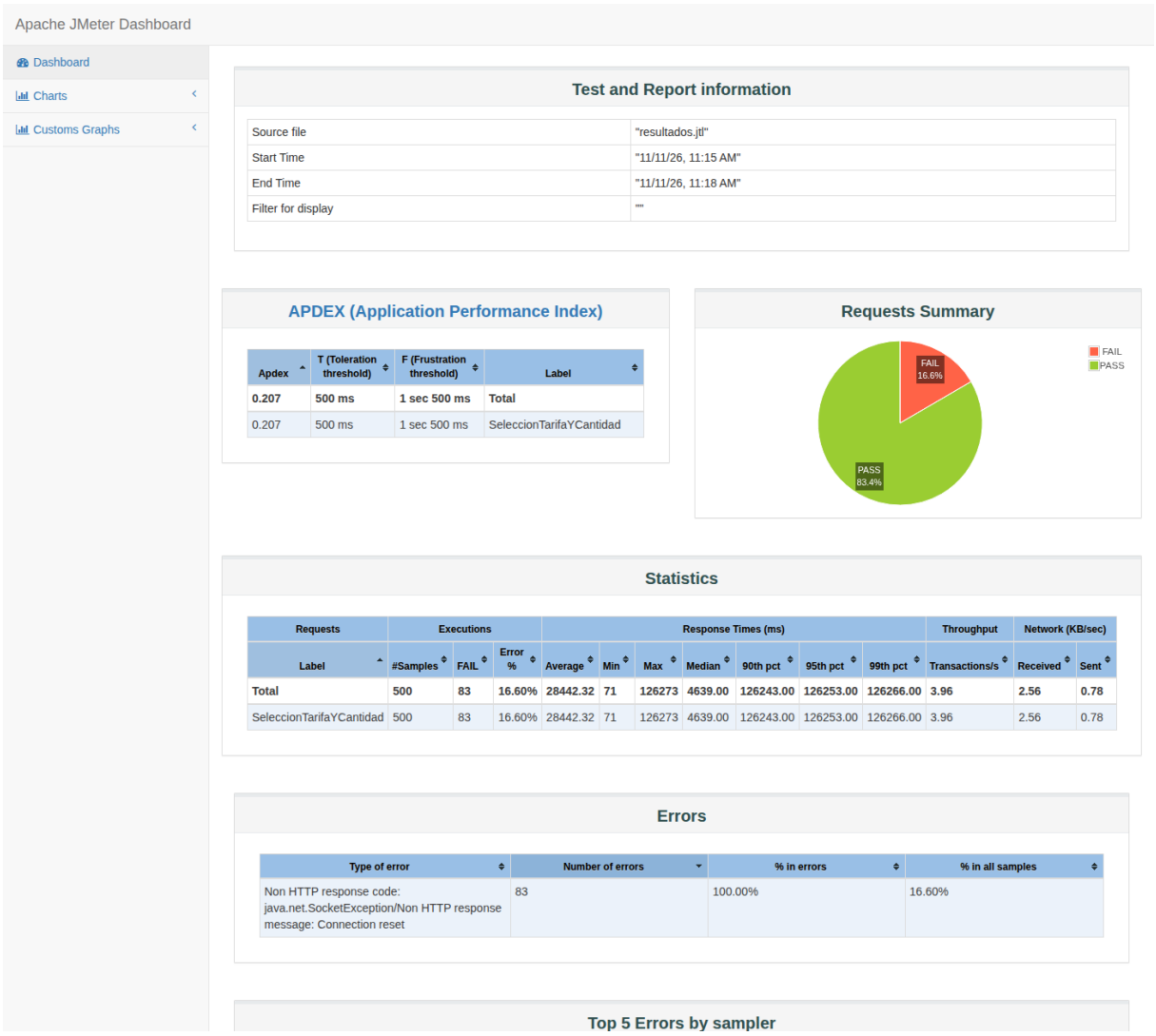
Configuración del escenario de la pregunta 8.

Resultados observados

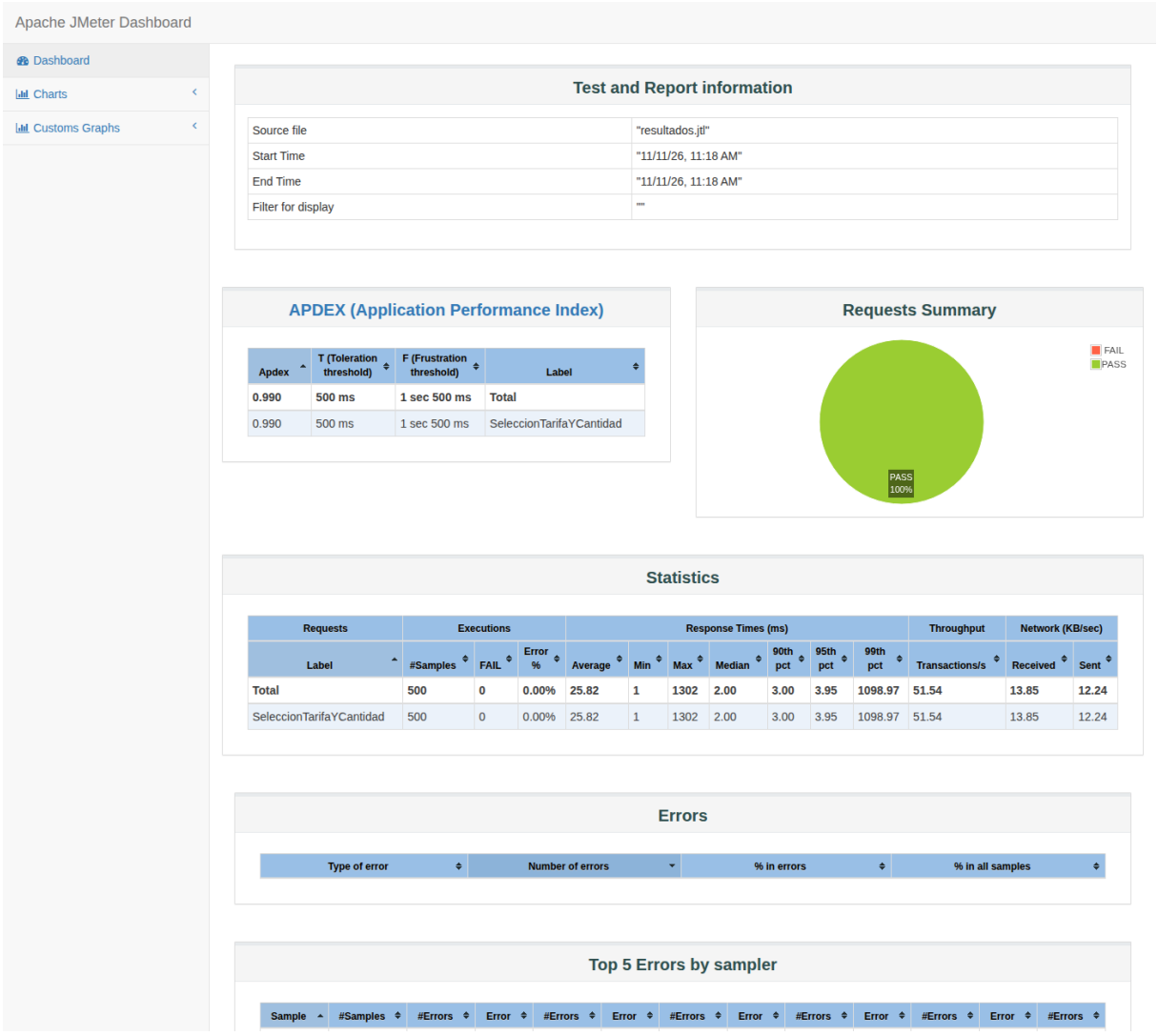
Escenario	Muestras	Mediana ok (ms)	p95 ok (ms)	Máx. ok (ms)	Fallidas	% Error
Sincronizado	500	3048	57626	62767	83	16.6 %
Distribuido	500	2	4	1302	0	0.0 %



Izquierda: peticiones enviadas por segundo. Derecha: tiempos de las peticiones exitosas (escala logarítmica).



Reporte de JMeter, escenario sincronizado.



Reporte de JMeter, escenario distribuido.

Mi respuesta

Cambia mucho. Con el Temporizador sincronizado las 500 peticiones salen en el mismo segundo y la API no lo soporta bien: 83 fallaron (16.6 %) con “Connection reset”, la mediana de las que sí respondieron fue de 3.0 s y el p95 de 58 s. Cuando los mismos 500 usuarios llegan repartidos en 10 s (unos 50 por segundo), ninguna falló, la mediana fue de 2 ms y el p95 de 4 ms.

La diferencia es lo que el temporizador sincronizado sirve para probar: un pico real de llegada simultánea, como cuando se abre la venta de boletas. La llegada distribuida es más suave y por eso esconde el problema. Si solo hubiera probado de forma distribuida, habría concluido que la API aguanta 500 usuarios sin errores.

Qué no pude medir o qué cambió el resultado

- La simulación de “al mismo tiempo” es aproximada: JMeter libera los hilos a la vez, pero llegan a la API con diferencias de milisegundos.
- Este escenario y el de la pregunta 7 se parecen. La diferencia es que aquí cada usuario hace una sola petición y en la 7 hace 10 seguidas.

Pregunta 9 · Concurrencia

¿Qué pasaría si… *el total cobrado no coincide con precio × cantidad bajo carga? ¿Cómo lo validarías con el CSV?*

Escenario

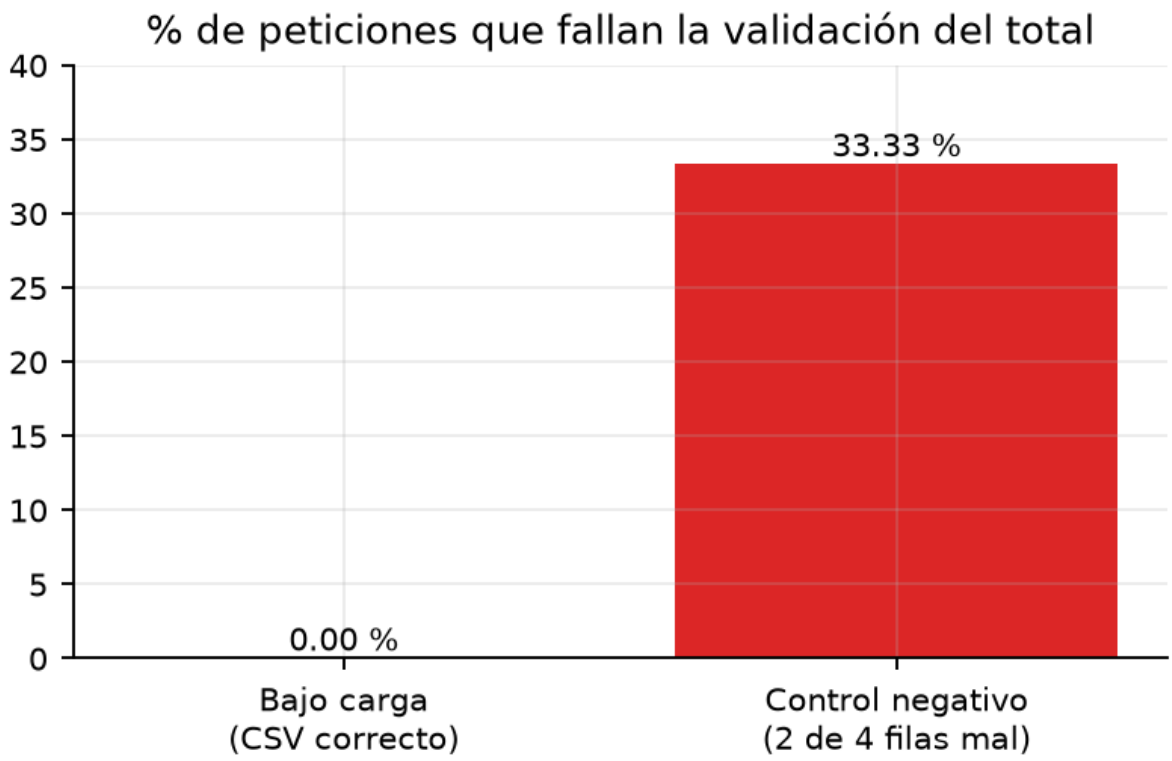
Parámetro	Valor
Hilos (usuarios)	200 (control negativo: 20)
Periodo de subida	10 s
Duración	60 s (bucle infinito)
Temporizador	100 ms + hasta 200 ms
CSV	datos.csv con 3 columnas: tarifa, cantidad, total_esperado (precio × cantidad)
Validación	JSON Assertion: . *total* = {total_esperado}, en Selección y en Pago
Control negativo	datos_totales_incorrectos.csv con 2 de 4 filas mal a propósito

Configuración del escenario de la pregunta 9.

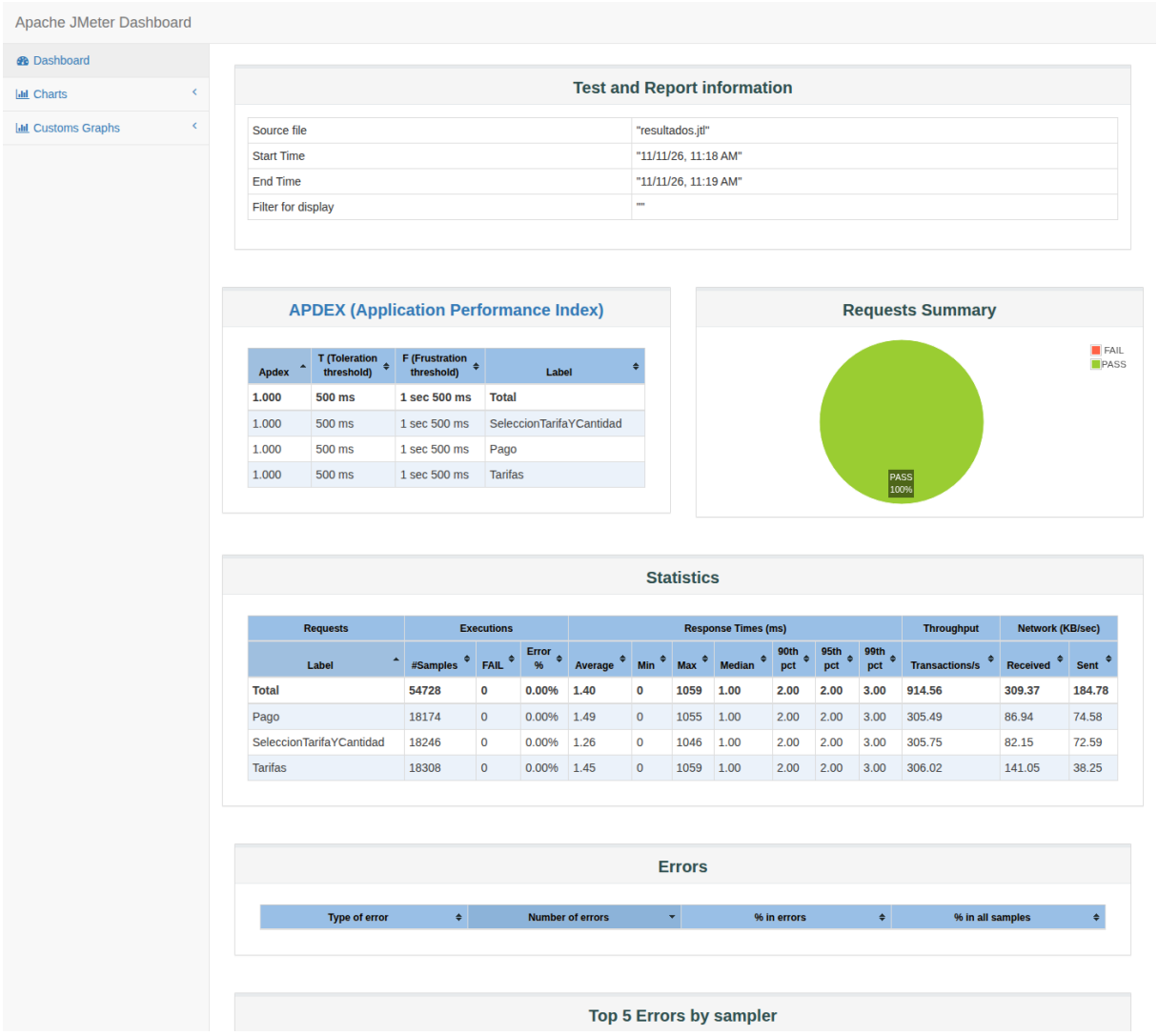
Resultados observados

Escenario	Muestras	Media (ms)	p95 (ms)	p99 (ms)	Máx (ms)	% Error	Rendimiento (peticiones/s)
Bajo carga (CSV correcto)	54 728	1.4	2	3	1 059	0.00 %	914.6
Control negativo	120	3.7	7	40	40	33.33 %	151.7

Dato	Valor
Validaciones del total bajo carga	18246 en Selección + 18174 en Pago, todas correctas
Control negativo	40 de 120 peticiones fallan: justo las de las 2 filas con total equivocado (10 + 10 en Selección, 20 en Pago)



La validación del total detecta los datos equivocados del control negativo y no falla bajo carga.



Reporte de JMeter bajo carga: 0 % de error.

Mi respuesta

Bajo carga no se encontró ninguna diferencia. En 18246 reservas y 18174 pagos con 200 usuarios simultáneos, el total fue siempre precio × cantidad. Ningún total quedó mezclado entre usuarios.

Cómo lo validé con el CSV: agregué al CSV una tercera columna, total_esperado, con el precio de la tarifa por la cantidad (por ejemplo, vip × 4 = 1 400 000). En JMeter puse una JSON Assertion con la ruta $.total y el valor ${total_esperado}. Así cada usuario compara lo que le devolvió la API contra lo que correspondía a su propia fila, y la comprobación vale aunque muchos usuarios compren a la vez.

Para estar seguro de que la validación sirve, hice un control negativo: un CSV con dos filas con el total mal a propósito. Fallaron exactamente 40 de las 120 peticiones: las de esas dos filas, tanto en Selección como en Pago. Si la aserción no funcionara, el escenario de carga habría dado 0 % de error igual.

Qué no pude medir o qué cambió el resultado

- La API de la profesora calcula el total con una multiplicación simple y sin estado compartido, así que no es sorprendente que no falle. Un error de concurrencia real aparecería con una API que reutilice datos entre peticiones.
- La validación solo mira los totales que responde la API; no comprueba que el valor guardado en la reserva sea el mismo (eso requeriría otro endpoint de consulta).

Pregunta 10 · Recursos

¿Qué pasaría si… *subo a 500 usuarios? ¿Cómo se comportan la memoria y la CPU durante la prueba? ¿Llegan a saturarse? ¿Cuánto cambia el tiempo de respuesta cuando eso pasa?*

Escenario

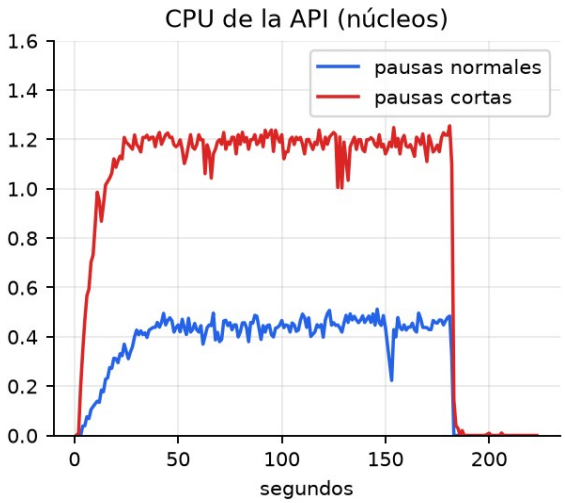
Parámetro	Valor
Hilos (usuarios)	500
Periodo de subida	30 s
Duración	180 s (bucle infinito)
Temporizador	normal: 500 ms + hasta 1000 ms / con presión: 50 ms + hasta 100 ms
Flujo	Tarifas → Selección → Pago
Monitoreo	CPU y memoria cada 1 s (equipo, JMeter y API)
Propiedades	-Jusuarios=500 -Jrampup=30 -Jduracion=180 (presión: -Jtimer_const=50 -Jtimer_rand=100)

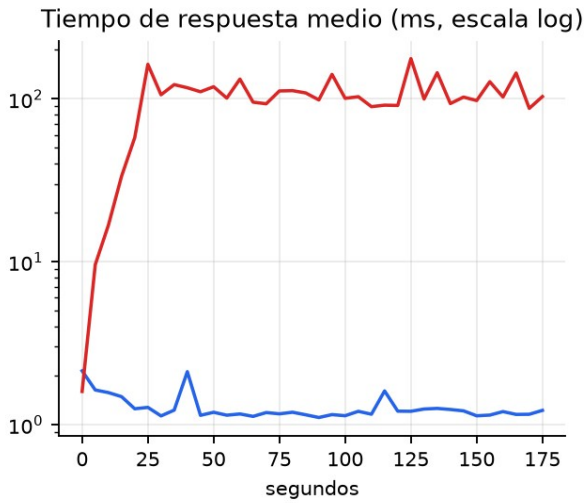
Configuración del escenario de la pregunta 10.

Resultados observados

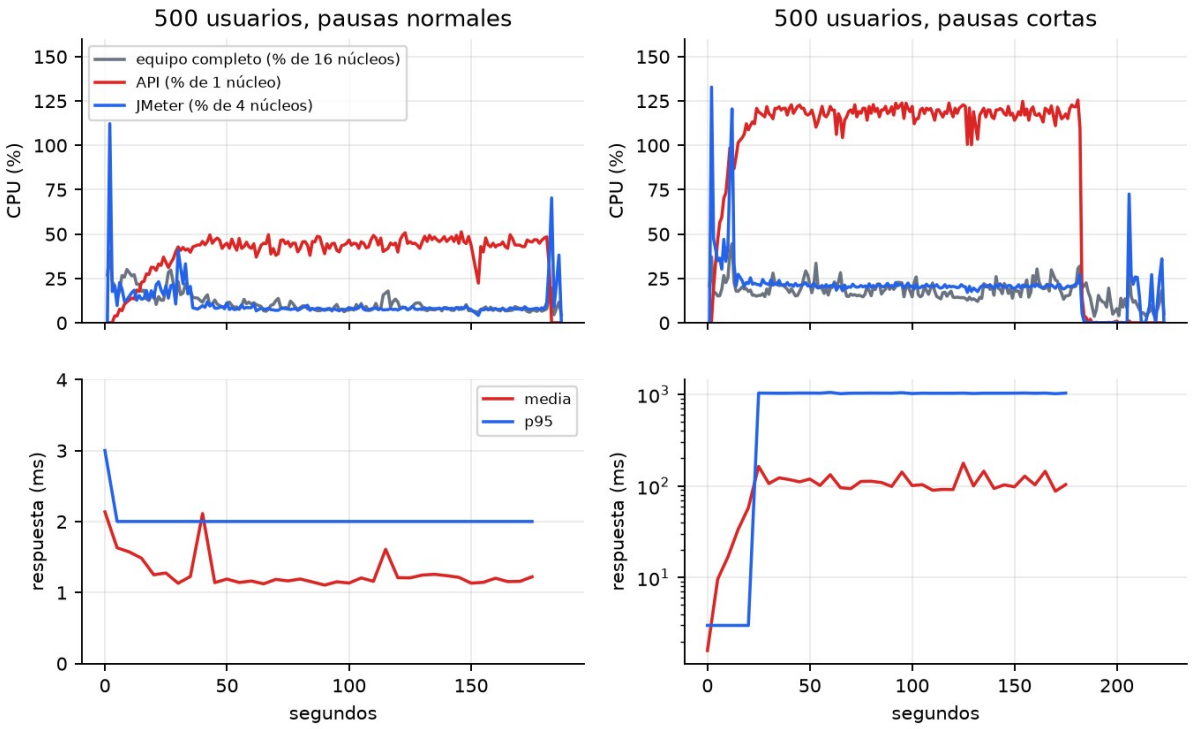
Escenario	Muestras	Media (ms)	p95 (ms)	p99 (ms)	Máx (ms)	% Error	Rendimiento (peticiones/s)
500 usuarios, pausas normales	81 291	1.2	2	3	1 063	0.00 %	453.3
500 usuarios, pausas cortas	407 796	103.7	1 022	2 074	135 385	0.00 %	2 005.8

Recurso (máximo)	Pausas normales	Pausas cortas
CPU del equipo (% de 16 núcleos)	40.3	44.5
CPU de JMeter (núcleos)	4.5	5.3
CPU de la API (núcleos)	0.51	1.25
Memoria del equipo (%)	61.4	59.7
Memoria de JMeter (MB)	1082.2	1174.2
Memoria de la API (MB)	22.6	23.1

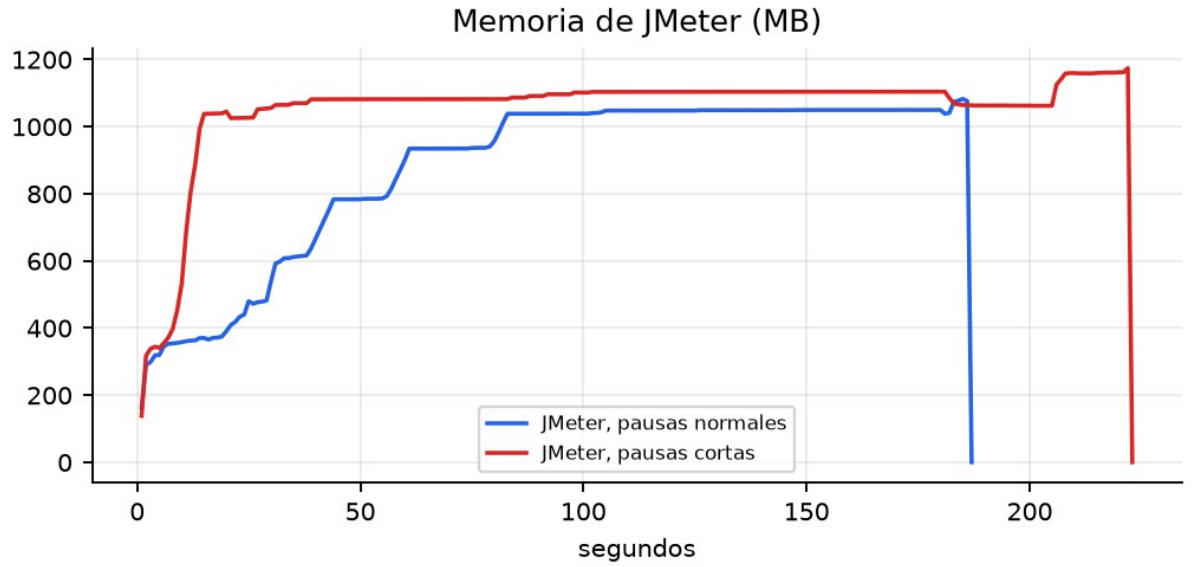




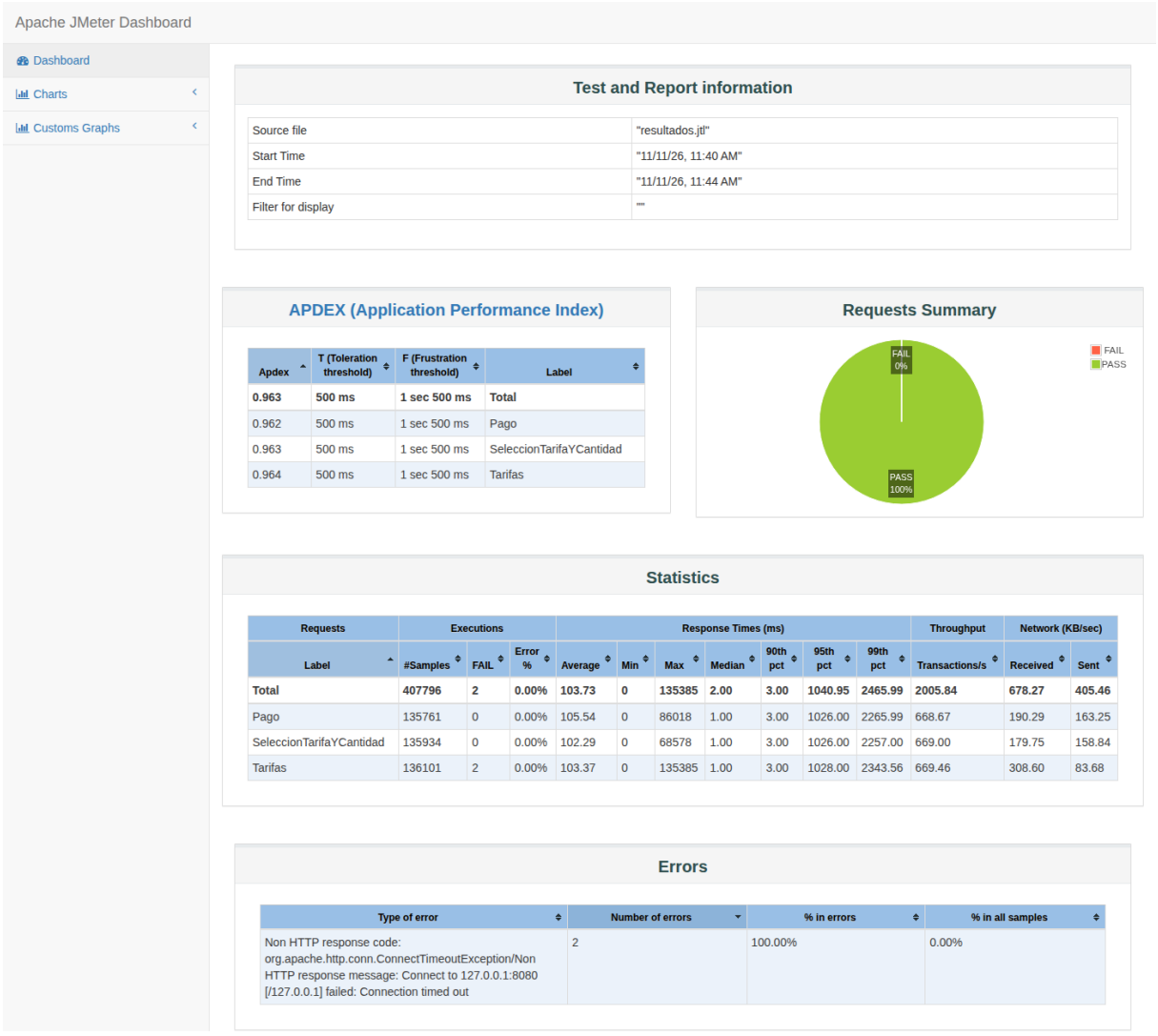
CPU de la API (izquierda) y tiempo de respuesta medio (derecha) con 500 usuarios.



Arriba: CPU. Abajo: tiempo de respuesta. A la izquierda, pausas normales; a la derecha, pausas cortas.



Memoria que usa JMeter durante la prueba.



Reporte de JMeter con 500 usuarios y pausas cortas.

Mi respuesta

Con las pausas normales no se saturó nada. El equipo usó como máximo un 40 % de CPU, JMeter hasta 4.5 núcleos y la API apenas 0.5. La memoria de la API se quedó en 22.6 MB y la de JMeter en cerca de 1 GB. El tiempo de respuesta se mantuvo en 1.2 ms de media.

Con pausas cortas, que sí empujan la carga a unas 2331 peticiones por segundo, la API llega a su límite: pasa de 0.5 a 1.3 núcleos, es decir, un poco más de un núcleo completo. Ahí el tiempo de respuesta cambia mucho: la media sube de 1.2 a 104 ms (unas 86 veces), el p95 pasa de 2 a 1022 ms y el máximo llega a 135 segundos.

Lo más interesante es que el CPU total del equipo nunca se vio saturado (máximo 44 %). El cuello de botella fue el proceso de la API, que usa algo más de un núcleo, y no el equipo completo. Por eso no basta con mirar el CPU del equipo: hay que mirar el proceso que se está probando. La memoria no fue un problema en ninguno de los dos casos.

Qué no pude medir o qué cambió el resultado

- No usé PerfMon con ServerAgent; el muestreador propio mide CPU y memoria del equipo, de JMeter y de la API cada segundo, pero no mide red ni disco.
- Como JMeter y la API comparten el equipo, el CPU del equipo mezcla a los dos. Por eso los separé por proceso.
- El escenario de “pausas cortas” lo agregué yo para poder observar la saturación: con las pausas normales la API no llegaba a su límite con 500 usuarios.

Conclusiones

- Con carga normal (pausas de 0,5 a 1,5 s) la API aguanta 500 usuarios sin problemas: el tiempo de respuesta no cambia y no hay errores (preguntas 1, 5 y 10).
- La API se satura cerca de las 2 300 peticiones por segundo. Antes de ese punto el throughput crece con los usuarios; después se estanca y los tiempos se disparan (preguntas 3, 4 y 10).
- El límite que se encontró es el de un solo proceso de la API, no del equipo completo. El CPU del equipo se quedó por debajo del 55 % (pregunta 10).
- Lo que más daña a la API es la llegada simultánea de muchas conexiones: 500 usuarios al mismo tiempo produjeron errores de “Connection reset” (16,6 % en la pregunta 8 y 0,66 % en la 7), mientras que repartidos nunca fallaron (preguntas 7 y 8).
- La lógica de negocio es correcta bajo carga: el total siempre fue precio × cantidad (pregunta 9).
- Las pausas definen qué estoy midiendo: con temporizadores simulo personas, sin ellos mido el límite de la API (pregunta 3).

Cómo reproducir

python3 scripts/correr_experimentos.py (todos los escenarios) o con el nombre de uno, por ejemplo q4-escalones. Los reportes HTML quedan en pruebas/experimentacion/resultados/<escenario>/reporte/index.html.